

12c Online Statistics Gathering & Session Private Statistics

12c came with two new features related to statistics gathering. In the first part I'll show some important effects to know before using Online Statistics Gathering – which are there by default. And then we will see how it combines with Session Private Statistics when we are on a Global Temporary Table for a transparent improvement of execution plans.

Frank Pachot, Senior Consultant, dbi Services

Partitioned table, local and global indexes, histograms

Here is my test case. In order to see all the behaviours, I create a partitioned table with local and global indexes and histograms:

```
SQL> create table DEMO_TABLE (user_id,username)
  2      partition by hash (user_id) partitions 2
  3      as select rownum,to_char(rownum) from dual connect by level<=1000
  4  /
Table created.

SQL> create index DEMO_LOCAL on DEMO_TABLE(user_id) local;
Index created.

SQL> create index DEMO_GLOBAL on DEMO_TABLE(username) global;
Index created.

SQL> exec dbms_stats.set_table_prefs(user,'DEMO_TABLE','METHOD_OPT','for all columns size 2');
PL/SQL procedure successfully completed.
```

And then I check the statistics.

Thanks to the 12 online statistics gathering, we have statistics for that newly created table:

```
SQL> select table_name, object_type, partition_name, num_rows, last_analyzed, global_stats, stale_stats from user_tab_statistics
where table_name='DEMO_TABLE' order by 1,3 nulls first;
```

TABLE_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	LAST_ANALYZED	GLOBAL_STATS	STALE_STATS
DEMO_TABLE	TABLE		1000	tue 17:53:48	YES	NO
DEMO_TABLE	PARTITION	SYS_P452			NO	
DEMO_TABLE	PARTITION	SYS_P453			NO	

But there are only global statistics and no statistics for the partitions.

The create index has gathered statistics, but that is not a new feature (it's the default since 10g):

```
SQL> select index_name, object_type, partition_name, num_rows, distinct_keys, last_analyzed, global_stats, stale_stats from user_
ind_statistics where table_name='DEMO_TABLE' order by 1,3 nulls first;
```

INDEX_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	DISTINCT_KEYS	LAST_ANALYZED	GLOBAL_STATS	STALE_STATS
DEMO_GLOBAL	INDEX		1000	1000	tue 17:53:48	YES	NO
DEMO_LOCAL	INDEX		1000	505	tue 17:53:48	NO	NO
DEMO_LOCAL	PARTITION	SYS_P454	495	495	tue 17:53:48	NO	
DEMO_LOCAL	PARTITION	SYS_P455	505	505	tue 17:53:48	NO	

We have statistics for the global index and for the local index and its partitions.

But be careful, there are no global statistics. The number of distinct keys is wrong here for the partitioned index at a global level. It shows 505, but I actually have 1000 distinct values. This has been described by Chris Antognini on his blog: <http://antognini.ch/2009/12/does-create-index-gather-global-statistics/>

I don't check column statistics yet because I've created the table preference afterwards.

The first conclusion is that online statistics gathering is a nice feature, but it is not sufficient to give an exhaustive input to the optimizer. We have to gather statistics. But there is the new GATHER AUTO function that helps to gather only the missing or stale statistics.

Gather Auto

Here is the GATHER AUTO call:

```
SQL> exec dbms_stats.gather_table_stats( ownname=>user, tabname=>'DEMO_TABLE', options =>'GATHER AUTO' );
PL/SQL procedure successfully completed.
```

And now let's look at the same statistics again:

```
SQL> select table_name, object_type, partition_name, num_rows, last_analyzed, global_stats, stale_stats from user_tab_statistics
where table_name='DEMO_TABLE' order by 1,3 nulls first;
```

TABLE_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	LAST_ANALYZED	GLOBAL	STALE
DEMO_TABLE	TABLE		1000	tue 17:53:48	YES	NO
DEMO_TABLE	PARTITION	SYS_P452	495	tue 17:56:20	YES	NO
DEMO_TABLE	PARTITION	SYS_P453	505	tue 17:56:20	YES	NO

Table statistics were not gathered again (it's the same LAST_ANALYZED as above), but now I have statistics for partitions, as well as global statistics. And here are the indexes:

```
SQL> select index_name, object_type, partition_name, num_rows, distinct_keys, last_analyzed, global_stats, stale_stats from user_ind_statistics
where table_name='DEMO_TABLE' order by 1,3 nulls first;
```

INDEX_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	DISTINCT_KEYS	LAST_ANALYZED	GLOBAL	STALE
DEMO_GLOBAL	INDEX		1000	1000	tue 17:56:20	YES	NO
DEMO_LOCAL	INDEX		1000	1000	tue 17:56:20	YES	NO
DEMO_LOCAL	PARTITION	SYS_P454	495	495	tue 17:56:20	YES	NO
DEMO_LOCAL	PARTITION	SYS_P455	505	505	tue 17:56:20	YES	NO

For the indexes I have the global statistics that were missing before; distinct keys is now accurate at the global level; 1000 distinct rows. And now the histograms. Remember I've set table preference to 'all columns size 2'.

```
SQL> select table_name, column_name, histogram, num_distinct, last_analyzed, global_stats from user_tab_col_statistics where table_name='DEMO_TABLE'
order by 1,3 nulls first;
```

TABLE_NAME	COLUMN_NAME	HISTOGRAM	NUM_DISTINCT	LAST_ANALYZED	GLOBAL
DEMO_TABLE	USER_ID	HYBRID	1000	tue 17:56:20	YES
DEMO_TABLE	USERNAME	HYBRID	1000	tue 17:56:20	YES

Histograms are calculated. As I have many distinct values it's an HYBRID histogram which replaces the 11g height balanced ones. If we look at the LAST_ANALYZED everything has been recalculated, except the table ones which were already accurate.

Truncate

Online statistics gathering occurs only when the table has no rows before the load and it was the case with the Create Table As Select. If I truncate the table, then the optimizer knows that the table is empty and Online Statistics Gathering can occur again.

```
SQL> truncate table DEMO_TABLE;
Table truncated.
```

And I have to bulk load with direct-path insert. As the table is already there, I do it with the APPEND hint.

```
SQL> insert /*+ append */ into DEMO_TABLE select 1, 'One' from
dual connect by level<=100;
100 rows created.
```

But note that I've inserted only 100 rows here. All with the same value (so only one distinct value). So how are my statistics now?

```
SQL> select table_name,object_type,partition_name,num_rows,last_analyzed,global_stats,stale_stats from user_tab_statistics where
table_name='DEMO_TABLE' order by 1,3 nulls first;
```

TABLE_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	LAST_ANALYZED	GLOBAL_STATS	STALE_STATS
DEMO_TABLE	TABLE		100	tue 17:57:36	YES	NO
DEMO_TABLE	PARTITION	SYS_P452	495	tue 17:56:20	YES	NO
DEMO_TABLE	PARTITION	SYS_P453	505	tue 17:56:20	YES	NO

As we already know the statistics were gathered only for the table and not for the partition.

But there is a problem here. Previously I had no statistics, which is not so bad because dynamic sampling occurs in that case and estimation can be reliable. But look, I still have my previous partition statistics, which are totally wrong now because they come from the 1000 rows table. It's wrong and inconsistent, the table shows 100 rows but partitions show 495+505 rows.

```
SQL> select index_name, object_type, partition_name, num_rows, distinct_keys, last_analyzed, global_stats, stale_stats from user_
ind_statistics where table_name='DEMO_TABLE' order by 1,3 nulls first;
```

INDEX_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	DISTINCT_KEYS	LAST_ANALYZED	GLOBAL_STATS	STALE_STATS
DEMO_GLOBAL	INDEX		1000	1000	tue 17:56:20	YES	YES
DEMO_LOCAL	INDEX		1000	1000	tue 17:56:20	YES	YES
DEMO_LOCAL	PARTITION	SYS_P454	495	495	tue 17:56:20	YES	NO
DEMO_LOCAL	PARTITION	SYS_P455	505	505	tue 17:56:20	YES	NO

It's the same problem with the indexes. Here I don't create nor rebuild the index so the statistics didn't change. The number of rows is wrong and distinct values is wrong.

```
SQL> select table_name, column_name, histogram, num_distinct, last_analyzed, global_stats from user_tab_col_statistics where
table_name='DEMO_TABLE' order by 1,3 nulls first;
```

TABLE_NAME	COLUMN_NAME	HISTOGRAM	NUM_DISTINCT	LAST_ANALYZED	GLOBAL_STATS
DEMO_TABLE	USER_ID	NONE	1	tue 17:57:36	YES
DEMO_TABLE	USERNAME	NONE	1	tue 17:57:36	YES

Histograms were not gathered, but at least my column statistics show the current situation about the distinct values.

Obviously, we need to gather statistics, again with GATHER AUTO. I don't reproduce the result here. After GATHER AUTO table, index and columns statistics are all correct. Note that Online Statistics Gathering occurs only after truncate (not delete) and insert append (not pl/sql bulk insert).

Insert into partition

So, Online Statistics Gathering occurs only for the table and not the partition when we insert into the table. Do you know that we can insert directly into a specific partition, to any row that goes into that partition? The main reason to do that comes from the locking behaviour. When you insert in direct path, the table has to be locked (TM exclusive) to prevent concurrent DML. This is mandatory because we bypass the buffer cache, which manages the shared access to the block. We can name the partition in the insert statement. But we can also use the syntax where we choose the partition with one value that goes into it.

I truncate only the partition that is the target for the value 0.

```
SQL> alter table DEMO_TABLE truncate partition for(0);
Table truncated.
```

And insert 200 rows into it:

```
SQL> insert /*+ append */ into DEMO_TABLE partition for(0) select 0, null from dual connect by level<=200;
200 rows created.
```

And again I check my statistics:

TABLE_NAME	OBJECT_TYPE	PARTITION_NAME	NUM_ROWS	LAST_ANALYZED	GLOBAL_STATS	STALE_STATS
DEMO_TABLE	TABLE		100	tue 17:58:20	NO	NO
DEMO_TABLE	PARTITION	SYS_P452	0	tue 17:58:20	YES	NO
DEMO_TABLE	PARTITION	SYS_P453	200	tue 17:58:45	YES	NO

Only one statistic has been updated (at 17:58:45) and it is the table partition. It's the target object that get the Online Statistics Gathering. And once again, before I run dbms_stats the statistics are inconsistent, table stats show 100 rows where we have 200 in one partition.

Global Temporary Tables

How do you gather statistics on Global Temporary Tables in 11g?

Obviously you don't want the automatic job to gather stats and put a '0 rows' as a cardinality. That's the worst we can do.

Estimations of no rows often lead to bad execution plans. You can choose to leave them with no statistics. Dynamic sampling will build an accurate execution plan. But that plan will be shared by other sessions having a different amount of rows. Or you can choose to gather statistics after you load it, but it's the same issue, the plan will be shared.

My preference in 11g is to set some manual statistics, chosen so that it gives an acceptable plan whatever the number of rows is. Usually a plan that is good for a lot of rows is not too bad for a few rows. The opposite is not true and often gives unwanted nested loops. I set the right statistics and a lock them so that no other job will gather them.

In 12c you have Session Private Statistics that you set with the GLOBAL_TEMP_TABLE_STATS preference. There are two nice consequences with them.

Let's see. I create a GTT and set the Private Session statistics for it:

```
SQL> create global temporary table DEMO_GTT(n number) on commit preserve rows;
Table created.

SQL> exec dbms_stats.set_table_prefs( user,'DEMO_GTT', 'GLOBAL_TEMP_TABLE_STATS', 'SESSION' );
PL/SQL procedure successfully completed.
```

Then I insert 1 row in direct path:

```
SQL> insert /*+ append */ into DEMO_GTT select rownum from dual connect by level<=1;
1 row created.

SQL> commit;

SQL> select count(*) from DEMO_GTT;

COUNT(*)
-----
1
```

Here is the execution plan with estimated rows and actual rows:

```
SQL> select * from table(dbms_xplan.display_cursor('allstats last'))

PLAN_TABLE_OUTPUT
-----
SQL_ID 1wpmvt9gd1p4b, child number 0
-----
select count(*) from DEMO_GTT
-----
| Id | Operation          | Name      | Starts | E-Rows | A-Rows |
|----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT   |           |      1 |        |      1 |
| 1  | SORT AGGREGATE     |           |      1 |        |      1 |
| 2  | TABLE ACCESS FULL| DEMO_GTT  |      1 |        |      1 |
-----
Note
----
- Global temporary table session private statistics used
```

Now I do the same in another session, but that time inserting 1000 rows. Note that I've never called dbms_stats and dynamic sampling does not occur. We have statistics because of Online Statistics Gathering as I'm doing a direct-path insert.

```
SQL> insert /*+ append */ into DEMO_GTT select rownum from dual connect by level<=1000;
1000 row created.

SQL> commit;
```

And here is the execution plan with estimated rows and actual rows:

```
PLAN_TABLE_OUTPUT
-----
SQL_ID 1wpmvt9gd1p4b, child number 1
-----
select count(*) from DEMO_GTT
-----
| Id | Operation          | Name      | Starts | E-Rows | A-Rows |
|----|-----|-----|-----|-----|-----|
| 0  | SELECT STATEMENT    |           |      1 |        |        |
| 1  | SORT AGGREGATE      |           |      1 |        |        |
| 2  | TABLE ACCESS FULL  | DEMO_GTT  |      1 |    1000 |    1000 |
-----
Note
-----
- Global temporary table session private statistics used
```

If you were in 11g you would see the same estimation as the first query, reusing the same plan. But now in 12c the estimation is relevant for my session with 1000 rows.

This is the first point with Session Private Statistics. They are stored only for your session so you will never see the number of rows calculated from another session. That is very good, if you don't share the data why would you share their statistics?

But there is another very interesting point. Have you seen that it's another child cursor?

Let's get the reason from v\$sql_shared_cursor:

```
SQL> select reason from v$sql_shared_cursor where sql_id='1wpmvt9gd1p4b'
REASON
-----
<ChildNode><ChildNumber>0</ChildNumber><ID>47</ID><reason>Session Specific Cursor Session Mismatch(1)</reason><size>7x2</size><session_mismatch>1</session_mismatch><current_qcseid>18</current_qcseid><current_qcsesid>37119</current_qcsesid><current_qcinst>1</current_qcinst><stored_qcseid>138</stored_qcseid><stored_qcsesid>5842</stored_qcsesid><stored_qcinst>1</stored_qcinst></ChildNode>
```

With Session Private Statistics the optimizer will not share an execution plan that was optimized for another session. A new child is created for the 'Session Specific Cursor Session Mismatch' reason. And a new execution plan is optimized.

And this is where Online Statistics Gathering is interesting because the solution is transparent in that case. If you have a use-case like this one where GTT is loaded by a direct-path insert, then you just have to set GLOBAL_TEMP_TABLE_STATS to 'SESSION', unlock the statistics if they were locked, and each session has the right execution plan. As we have seen above, not all statistics are gathered, but that may be sufficient for a temporary table. And because statistics are private, we don't have the inconsistency we have seen above with Online Statistics Gathering on permanent tables.

GTT on commit

A final note about those 12c changes in statistics gathering on GTT. In 11g the dbms_stats did a commit at the start. So if you did gather stats after the load, you had to set the GTT as ON COMMIT PRESERVE ROWS. Or you just vacuum what you've loaded. That has changed in 12c. If you now choose to do a conventional insert followed by dbms_stats (having set private stats of course) then you don't need to set on commit preserve rows anymore.

However, if you choose the previous solution, which is a direct-path insert that implicitly gather statistics, then you still have to be ON COMMIT PRESERVE ROWS. The reason is that after the direct-path insert you have to commit before reading the table.

Conclusion

Online Statistics Gathering has several side effects that we need to know. We have seen that we still need to gather statistics. Session Private Statistics has also its drawbacks, a lot of applications are using a connection pool, reusing a previous session. So even if the statistics are private to the session then they can be used in another context.

But the combination of those two features is great when working on Global Temporary Tables. When used properly they can help to improve execution plans on GTT without changing the code. Let the insert automatically gather statistics and let the optimizer automatically avoid reusing an unadapted plan. ■



ABOUT THE AUTHOR

Franck Pachot
Senior Consultant, dbi services

Franck Pachot is senior consultant at dbi services in Switzerland. He has 20 years of experience in Oracle databases, all areas from development, data modeling, performance, administration, training. He tries to leverage knowledge sharing in forums, publications, presentations, and became recently an Oracle Certified Master.